

PROGRAMAÇÃO DE COMPUTADORES



Listas, Tuplas, Vetores, Matrizes e Arrays

Prof. Dr. José Jairo de Santana e Silva

Aula sobre Estruturas Sequenciais

Listas, Tuplas, Vetores, Matrizes e Arrays

AGENDA DA AULA

☰ Agenda

Estruturas sequenciais em Python

- Conceito de sequência
- Listas: criação, índices, fatiamento, métodos
- Mutabilidade e cópias de lista
- Tuplas: imutabilidade e desempacotamento
- Vetores como listas unidimensionais
- Matrizes como listas de listas
- Arrays do NumPy: visão inicial
- Exercícios guiados em sala
- Lista de exercícios e desafios

OBJETIVOS DE APRENDIZAGEM

🎯 Objetivos

Ao final desta aula, o estudante será capaz de:

- Criar e manipular **listas** com indexação, fatiamento e métodos
- Compreender **mutabilidade** e diferenciar listas de tuplas
- Usar **tuplas** para dados que não devem mudar
- Tratar listas como **vetores** unidimensionais e operar sobre eles
- Representar **matrizes** como listas de listas e iterar com **for** aninhado
- Reconhecer o que é um **array do NumPy** e por que ele existe
- Combinar essas estruturas com condicionais, laços e funções nativas

O QUE É UMA SEQUÊNCIA?

Sequência: ideia geral

Uma **sequência** é uma coleção de elementos **ordenados**, onde cada elemento ocupa uma **posição**.

Você já trabalhou com sequências:

- Strings: **"Python"** é uma sequência de caracteres
- **range(5)** é uma sequência de inteiros

Hoje vamos conhecer mais duas sequências fundamentais:

Estrutura	Pode mudar?	Sintaxe
Lista	Sim (mutável)	[1, 2, 3]
Tupla	Não (imutável)	(1, 2, 3)

Toda sequência tem **tamanho** (**len**), permite **indexação** e pode ser percorrida com **for**.

Como o Python conta posições

índice positivo:	0	1	2	3	4
	+-----+-----+-----+-----+-----+				
	10	20	30	40	50
	+-----+-----+-----+-----+-----+				
índice negativo:	-5	-4	-3	-2	-1

- Sempre começa em 0 na primeira posição
- Índices negativos começam pelo fim (-1 é o último)

Esta convenção vale para strings, listas, tuplas e qualquer sequência indexável.

LISTAS

☰ Lista: sequência mutável de posições

Uma **lista** organiza vários valores em uma única variável. Cada elemento ocupa uma **posição** e pode ser acessado pelo índice.

```
nomes = ["Ana", "Bruno", "Carla"]
```

índice:	0	1	2
valor:	"Ana"	"Bruno"	"Carla"
índice:	-3	-2	-1

Operações comuns:

```
nomes[1] = "Diego"      # troca um valor
nomes.append("Eva")     # adiciona no final
nomes.insert(0, "Bia")  # insere em uma posição
nomes.remove("Carla")   # remove pelo valor
```

Use listas quando a coleção precisa **crescer**, **diminuir** ou ter elementos **substituídos**.

Criando listas

Listas são criadas com **colchetes** []. Podem conter qualquer tipo de dado.

Código:

```
notas = [7.5, 8.0, 6.5, 9.0]
nomes = ["Ana", "Bruno", "Carla"]
mista = [1, "texto", 3.14, True]
vazia = []

print(notas)
print(nomes)
print(mista)
print(vazia)
```

Saída:

```
## [7.5, 8.0, 6.5, 9.0]
## ['Ana', 'Bruno', 'Carla']
## [1, 'texto', 3.14, True]
## []
```

Em Python, uma lista pode misturar tipos. Em geral, prefira manter o **mesmo tipo** para clareza.

⚙️ Indexação

Acesso a elementos por **posição** entre colchetes.

Código:

```
cores = ["vermelho", "verde", "azul", "amarelo"]  
  
print(cores[0])      # primeira  
print(cores[2])      # terceira  
print(cores[-1])     # última  
print(cores[-2])     # penúltima
```

Saída:

```
## vermelho  
## azul  
## amarelo  
## azul
```

■ Tentar acessar uma posição inexistente gera o erro **IndexError**.

✂ Fatiamento (slicing)

Pegar um **pedaço** da lista usando `[inicio:fim]`. O **fim não** entra no resultado.

Código:

```
nums = [10, 20, 30, 40, 50, 60]

print(nums[1:4])      # do índice 1 ao 3
print(nums[:3])       # do começo até o índice 2
print(nums[3:])       # do índice 3 até o fim
print(nums[::2])       # de 2 em 2
print(nums[::-1])     # invertido
```

Saída:

```
## [20, 30, 40]
## [10, 20, 30]
## [40, 50, 60]
## [10, 30, 50]
## [60, 50, 40, 30, 20, 10]
```

■ Sintaxe completa: `lista[inicio:fim:passo]`.

 Pergunta rápida

Qual a saída?

```
letras = ["a", "b", "c", "d", "e"]  
print(letras[1:4])  
print(letras[-2])  
print(letras[::-1])
```

Saída:

```
## ['b', 'c', 'd']
```

```
## d
```

```
## ['e', 'd', 'c', 'b', 'a']
```

Código junto: acessos e fatias

Digite e execute:

```
x = ["Ana", "Bruno", "Carla", "Diego", "Eva"]

print("Primeiro:", x[0])
print("Último:", x[-1])
print("Meio:", x[1:4])
print("Invertida:", x[::-1])

x[1] = "Beatriz"
x.append("Felipe")
print("Atualizada:", x)
```

Depois altere:

- Troque o último nome por outro nome
- Mostre apenas os três primeiros elementos
- Mostre os elementos de duas em duas posições

Listas são mutáveis

Podemos **alterar** elementos diretamente pelo índice.

Código:

```
frutas = ["maçã", "banana", "uva"]  
print("Antes:", frutas)  
  
frutas[1] = "pera"  
print("Depois:", frutas)
```

Saída:

```
## Antes: ['maçã', 'banana', 'uva']
```

```
## Depois: ['maçã', 'pera', 'uva']
```

Esta é a principal diferença entre listas e tuplas (que veremos a seguir).

+ Adicionar elementos

Código:

```
pilha = []

pilha.append(10)      # adiciona no final
pilha.append(20)
pilha.append(30)
print("Após appends:", pilha)

pilha.insert(1, 99)    # insere na posição 1
print("Após insert:", pilha)

pilha.extend([40, 50]) # adiciona vários no final
print("Após extend:", pilha)
```

Saída:

```
## Após appends: [10, 20, 30]
```

```
## Após insert: [10, 99, 20, 30]
```

```
## Após extend: [10, 99, 20, 30, 40, 50]
```

➤ Remover elementos

Código:

```
itens = ["a", "b", "c", "d", "e"]

itens.remove("c")      # remove pelo valor
print("Após remove:", itens)

removido = itens.pop()  # remove e devolve o último
print("Após pop:", itens, "| removido:", removido)

removido = itens.pop(0) # remove e devolve o do índice 0
print("Após pop(0):", itens, "| removido:", removido)

del itens[0]           # remove pelo índice
print("Após del:", itens)
```

Saída:

```
## Após remove: ['a', 'b', 'd', 'e']
```

```
## Após pop: ['a', 'b', 'd'] | removido: e
```

```
## Após pop(0): ['b', 'd'] | removido: a
```

```
## Após del: ['d']
```

Métodos úteis de lista (1/2)

Métodos que **alteram** a própria lista:

Método	Exemplo com x	O que faz	Resultado
<code>append()</code>	<code>x = [1, 2]</code> <code>x.append(3)</code>	Adiciona 3 no final	<code>x = [1, 2, 3]</code>
<code>insert()</code>	<code>x = [1, 3]</code> <code>x.insert(1, 2)</code>	Insere 2 na posição 1	<code>x = [1, 2, 3]</code>
<code>extend()</code>	<code>x = [1, 2]</code> <code>x.extend([3, 4])</code>	Junta outra lista ao final	<code>x = [1, 2, 3, 4]</code>
<code>remove()</code>	<code>x = [1, 2, 2, 3]</code> <code>x.remove(2)</code>	Remove o primeiro 2	<code>x = [1, 2, 3]</code>
<code>pop()</code>	<code>x = [1, 2, 3]</code> <code>y = x.pop()</code>	Remove e devolve o último	<code>x = [1, 2]</code> <code>y = 3</code>

Estes métodos mudam o conteúdo de `x`.

☰ Métodos úteis de lista (2/2)

Métodos que **ordenam**, **consultam** ou **copiam**:

Método	Exemplo com x	O que faz	Resultado
<code>sort()</code>	<code>x = [3, 1, 2]</code> <code>x.sort()</code>	Ordena a própria lista	<code>x = [1, 2, 3]</code>
<code>reverse()</code>	<code>x = [1, 2, 3]</code> <code>x.reverse()</code>	Inverte a própria lista	<code>x = [3, 2, 1]</code>
<code>index()</code>	<code>x = ["a", "b", "c"]</code> <code>x.index("b")</code>	Retorna a posição de "b"	1
<code>count()</code>	<code>x = ["a", "b", "a"]</code> <code>x.count("a")</code>	Conta quantas vezes "a" aparece	2
<code>copy()</code>	<code>x = [1, 2]</code> <code>y = x.copy()</code> <code>y.append(3)</code>	Cria uma cópia independente	<code>x = [1, 2]</code> <code>y = [1, 2, 3]</code>

| `sort()` e `reverse()` também alteram `x`. `index()` e `count()` apenas devolvem um valor.

Iterando com **for**

Duas maneiras comuns de percorrer uma lista:

Por **valor**:

```
notas = [6.5, 8.0, 9.5, 4.0, 7.0]
for nota in notas:
    print(nota)
```

```
## 6.5
## 8.0
## 9.5
## 4.0
## 7.0
```

Por **índice e valor**, com **enumerate**:

```
for i, nota in enumerate(notas, start=1):
    print(f"Aluno {i}: {nota}")
```

```
## Aluno 1: 6.5
## Aluno 2: 8.0
## Aluno 3: 9.5
## Aluno 4: 4.0
## Aluno 5: 7.0
```

Q in para verificar pertencimento

```
times = ["Coritiba", "Athletico", "Paraná"]  
  
print("Coritiba" in times)  
print("Santos" in times)  
print("Santos" not in times)
```

```
## True
```

```
## False
```

```
## True
```

■ Muito útil em condicionais: `if "Coritiba" in times: ...`

⚠ Cuidado com cópias

Atribuir uma lista **não cria uma cópia**, apenas dá outro nome ao mesmo objeto.

```
a = [1, 2, 3]
b = a          # NÃO é cópia
b.append(99)

print("a:", a)
print("b:", b)

# Para fazer uma cópia real:
c = a.copy()   # ou list(a) ou a[:]
c.append(7)
print("a:", a)
print("c:", c)
```

```
## a: [1, 2, 3, 99]
## b: [1, 2, 3, 99]
```

```
## a: [1, 2, 3, 99]
## c: [1, 2, 3, 99, 7]
```

Erro clássico de iniciante. Quando precisar **modificar sem afetar a original**, use `.copy()`.

Pergunta rápida

Qual a saída?

```
nums = [3, 1, 4, 1, 5]
nums.append(9)
nums.remove(1)
nums.sort()
print(nums)
print(len(nums))
```

Saída:

```
## [1, 3, 4, 5, 9]
## 5
```

Atenção: `remove(1)` tira **só a primeira** ocorrência do valor **1**.

PAUSA PRÁTICA: LISTAS

 Exercícios da seção: listas

1. Crie uma lista com 6 números. Mostre o primeiro, o último, o tamanho, a soma e a média.
2. Leia 5 nomes pelo teclado e guarde em uma lista. Depois mostre a lista em ordem inversa.
3. Dada a lista $x = [4, 7, 2, 7, 9, 2, 1]$, remova a primeira ocorrência de 7, adicione 10 ao final e ordene a lista.
4. Crie uma lista apenas com os números pares de 1 a 30.

■ Ao terminar, compare com o colega ao lado: o resultado é igual? O caminho foi igual?

TUPLAS

O que é uma tupla

Tupla é uma sequência **imutável**, criada com **parênteses ()**.

Código:

```
ponto = (3, 5)
cores_rgb = (255, 128, 0)
dia = ("segunda", "terça", "quarta")

print(ponto)
print(cores_rgb)
print(dia[0])
print(len(dia))
```

Saída:

```
## (3, 5)
## (255, 128, 0)
## segunda
## 3
```

Indexação e fatiamento funcionam **igualzinho** ao das listas.

🚫 Imutabilidade

Não é possível alterar uma tupla depois de criada.

```
ponto = (3, 5)
ponto[0] = 10      # ERRO: TypeError
ponto.append(7)    # ERRO: tuplas não têm append
```

Por que isso é útil?

- Garante que dados importantes não sejam alterados por engano
- Pode ser usada como **chave de dicionário** (listas não podem)
- Comunica a intenção: "isso aqui não muda"

Tupla com **um único elemento** precisa da vírgula: `(7,)`. Sem ela, `(7)` é apenas o número entre parênteses.

Código junto: tupla como registro

Digite e execute:

```
aluno = ("Ana", "2026001", 8.5)

nome, matricula, nota = aluno

print("Nome:", nome)
print("Matrícula:", matricula)
print("Nota:", nota)

if nota >= 7:
    print("Situação: aprovado")
else:
    print("Situação: reprovado")
```

Depois altere:

- Troque os dados do aluno
- Crie uma segunda tupla para outro aluno
- Tente alterar `aluno[2]` e observe o erro

Desempacotamento

Atribuir vários nomes de uma vez a partir de uma sequência.

Código:

```
ponto = (3, 5)
x, y = ponto
print("x =", x, "| y =", y)

# Também funciona com listas
notas = [7.5, 8.0, 6.5]
n1, n2, n3 = notas
print(n1, n2, n3)

# Troca de variáveis
a, b = 10, 20
a, b = b, a
print("a =", a, "| b =", b)
```

Saída:

```
## x = 3 | y = 5
```

```
## 7.5 8.0 6.5
```

```
## a = 20 | b = 10
```


Lista vs Tupla

Característica	Lista []	Tupla ()
Pode mudar?	Sim	Não
Velocidade	Mais lenta	Mais rápida
Métodos	Muitos	Poucos
Boa para...	Coleções que mudam	Dados fixos
Exemplo típico	Lista de notas	Coordenadas (x, y)

Regra prática:

Se a coleção precisa **crescer ou mudar**, use lista. Se representa um **registro fixo**, prefira tupla.

PAUSA PRÁTICA: TUPLAS

 Exercícios da seção: tuplas

1. Crie uma tupla `produto = ("Caderno", 12.90, 5)`. Use desempacotamento para mostrar nome, preço e quantidade.
2. Crie uma lista de tuplas com 3 produtos no formato `(nome, preco)`. Percorra a lista com `for` e mostre apenas os produtos acima de R\$ 50.
3. Crie uma tupla `ponto = (3, 4)`. Use desempacotamento e calcule a distância até a origem: $(x^2 + y^2)^{0.5}$.

Regra da seção: se o dado representa um registro fixo, pense em tupla.

VETORES

Vetor: lista numérica em uma dimensão

Um **vetor** é uma sequência de valores numéricos em uma única dimensão. Em Python básico, podemos representá-lo com uma lista.

```
v = [10, 20, 30, 40, 50]
```

índice:	0	1	2	3	4
valor:	10	20	30	40	50

O que muda em relação a uma lista qualquer:

- Os elementos costumam ser do **mesmo tipo**
- A posição tem significado matemático
- Operações típicas: soma, média, máximo, mínimo, distância

Lista é a estrutura em Python. **Vetor** é a interpretação matemática dessa lista quando ela representa dados numéricos ordenados.

→ O que é um vetor

Em programação, **vetor** é uma sequência de valores **do mesmo tipo**, indexada de 0 a **$n-1$** .

Em Python, usamos uma **lista** para representar vetores 1D:

```
v = [10, 20, 30, 40, 50]
```

- **$v[0]$** é o primeiro elemento
- **$\text{len}(v)$** é o tamanho
- **$v[i]$** é o **i** -ésimo elemento

▮ Vetor é um conceito; lista é a estrutura concreta que usaremos.

Operações sobre vetores

Código:

```
v = [4, 8, 15, 16, 23, 42]

print("Tamanho:", len(v))
print("Soma:", sum(v))
print("Média:", sum(v) / len(v))
print("Maior:", max(v))
print("Menor:", min(v))
```

Saída:

Tamanho: 6

Soma: 108

Média: 18.0

Maior: 42

Menor: 4

Funções nativas que vimos na aula passada **resolvem boa parte** das tarefas com vetores.

+ Construindo um vetor com **for**

Código:

```
quadrados = []  
for i in range(1, 11):  
    quadrados.append(i * i)  
  
print(quadrados)  
print("Soma:", sum(quadrados))
```

Saída:

```
## [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
## Soma: 385
```

▮ Padrão clássico: criar lista vazia, percorrer com **for**, ir acumulando com **append**.

 Código junto: vetor de notas

Digite e execute:

```
notas = []

for i in range(1, 6):
    nota = float(input(f"Nota {i}: "))
    notas.append(nota)

media = sum(notas) / len(notas)

print("Notas:", notas)
print("Média:", round(media, 2))
print("Maior:", max(notas))
print("Menor:", min(notas))
```

Depois altere: conte quantas notas ficaram acima ou iguais à média.

⚠ Cuidado: listas não somam como vetores

Em Python puro, `+` entre listas **concatena** em vez de somar elemento a elemento.

```
a = [1, 2, 3]
b = [10, 20, 30]
print(a + b)
```

```
## [1, 2, 3, 10, 20, 30]
```

Para somar **elemento a elemento**, escrevemos um **for**:

```
soma = []
for i in range(len(a)):
    soma.append(a[i] + b[i])
print(soma)
```

```
## [11, 22, 33]
```

⌋ Mais adiante veremos que o **NumPy** já entrega esse tipo de soma "de graça".

Exemplo: produto escalar

Produto escalar entre dois vetores é a soma dos produtos elemento a elemento.

Código:

```
a = [1, 2, 3, 4]
b = [10, 20, 30, 40]

produto = 0
for i in range(len(a)):
    produto += a[i] * b[i]

print("Produto escalar:", produto)
```

Saída:

```
## Produto escalar: 300
```

Exemplo típico em álgebra linear, computação gráfica e processamento de sinais.

PAUSA PRÁTICA: VETORES

 Exercícios da seção: vetores

1. Leia 6 números e guarde em um vetor. Mostre a média e uma nova lista apenas com os valores acima da média.
2. Dadas as listas $\mathbf{a} = [2, 4, 6]$ e $\mathbf{b} = [1, 3, 5]$, crie uma lista com a multiplicação elemento a elemento.
3. Calcule o produto escalar de $\mathbf{a}$ e $\mathbf{b}$.
4. Crie um vetor com os quadrados dos números de 1 a 20.

■ Nesta seção, a pergunta central é: qual cálculo preciso fazer sobre cada posição?

MATRIZES

Matriz: lista de linhas

Uma **matriz** organiza valores em duas dimensões: **linhas** e **colunas**. Em Python puro, representamos uma matriz como uma lista em que cada elemento é outra lista.

	coluna 0	coluna 1	coluna 2
linha 0:	[1]	[2]	[3]
linha 1:	[4]	[5]	[6]
linha 2:	[7]	[8]	[9]

Para localizar um valor, usamos **duas coordenadas**:

- o primeiro índice escolhe a **linha**
- o segundo índice escolhe a **coluna**
- em Python puro: **M[linha][coluna]**

Uma matriz é uma **lista de linhas**. Cada linha é uma lista comum.

Matriz como lista de listas

Uma matriz é uma estrutura 2D com **linhas** e **colunas**. Em Python, representamos como **lista de listas**.

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

- `matriz[0]` é a primeira linha: `[1, 2, 3]`
- `matriz[0][2]` é o elemento da linha 0, coluna 2: 3
- `len(matriz)` é o número de linhas
- `len(matriz[0])` é o número de colunas

▮ Cada linha é, por sua vez, uma lista comum.

 Acessando elementos

Código:

```
M = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print("M[0]:", M[0])  
print("M[0][0]:", M[0][0])  
print("M[1][2]:", M[1][2])  
print("M[2][0]:", M[2][0])  
print("Linhas:", len(M))  
print("Colunas:", len(M[0]))
```

Saída:

```
## M[0]: [1, 2, 3]
```

```
## M[0][0]: 1
```

```
## M[1][2]: 6
```

```
## M[2][0]: 7
```

Iterando uma matriz

Por linha:

```
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
for linha in M:  
    print(linha)
```

```
## [1, 2, 3]
```

```
## [4, 5, 6]
```

```
## [7, 8, 9]
```

Por **linha e coluna**, com **for** aninhado:

```
for i in range(len(M)):  
    for j in range(len(M[i])):  
        print(f"M[{i}][{j}] = {M[i][j]}")
```

↳ Iteração aninhada: saída

Saída:

```
## M[0][0] = 1
## M[0][1] = 2
## M[0][2] = 3
## M[1][0] = 4
## M[1][1] = 5
## M[1][2] = 6
## M[2][0] = 7
## M[2][1] = 8
## M[2][2] = 9
```

┆ O **for** externo percorre **linhas**; o interno percorre **colunas** daquela linha.

 Código junto: matriz de notas

Digite e execute:

```
notas = [  
    [8.0, 7.5, 9.0],  
    [5.0, 6.0, 6.5],  
    [9.5, 8.5, 10.0]  
]  
  
for i in range(len(notas)):  
    media = sum(notas[i]) / len(notas[i])  
    print(f"Aluno {i + 1}: média = {media:.2f}")
```

Depois altere: mostre também a maior nota de cada aluno.

Soma de todos os elementos

Código:

```
M = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
total = 0  
for linha in M:  
    for valor in linha:  
        total += valor  
  
print("Soma total:", total)  
  
# Forma mais curta usando sum  
total2 = sum(sum(linha) for linha in M)  
print("Soma total (alternativa):", total2)
```

Saída:

```
## Soma total: 45
```

```
## Soma total (alternativa): 45
```

⊕ Matriz preenchida com zeros

Padrão muito comum: criar uma matriz $n \times m$ cheia de zeros antes de preencher.

```
linhas = 3
colunas = 4

M = []
for i in range(linhas):
    linha = []
    for j in range(colunas):
        linha.append(0)
    M.append(linha)

for linha in M:
    print(linha)
```

```
## [0, 0, 0, 0]
## [0, 0, 0, 0]
## [0, 0, 0, 0]
```

↔ Exemplo: tabuada como matriz

Código:

```
n = 5
tabuada = []

for i in range(1, n + 1):
    linha = []
    for j in range(1, n + 1):
        linha.append(i * j)
    tabuada.append(linha)

for linha in tabuada:
    print(linha)
```

Saída:

```
## [1, 2, 3, 4, 5]
## [2, 4, 6, 8, 10]
## [3, 6, 9, 12, 15]
## [4, 8, 12, 16, 20]
## [5, 10, 15, 20, 25]
```

 Pergunta rápida

Dada a matriz abaixo, qual a saída?

```
M = [[2, 4, 6], [1, 3, 5], [7, 8, 9]]  
print(M[1])  
print(M[1][2])  
print(M[2][0] + M[0][1])
```

Saída:

```
## [1, 3, 5]
```

```
## 5
```

```
## 11
```


PAUSA PRÁTICA: MATRIZES

 Exercícios da seção: matrizes

1. Dada uma matriz 3×3 , imprima apenas a diagonal principal.
2. Calcule a soma de cada linha de uma matriz 3×3 .
3. Calcule a soma de cada coluna de uma matriz 3×3 .
4. Crie uma matriz 4×4 preenchida com zeros. Depois altere a diagonal principal para 1.
 - Antes de programar, marque no papel quais índices representam linha e coluna.

ARRAYS DO NUMPY

Array NumPy: bloco numérico com formato

Um **array NumPy** é uma estrutura numérica com formato definido (**shape**). Ele pode representar vetores, matrizes e estruturas com mais dimensões.

Objeto	Shape	Acesso
vetor 1D	(5,)	v[i]
matriz 2D	(3, 4)	M[i, j]
array 3D	(2, 3, 4)	A[k, i, j]

A diferença principal não é apenas guardar dados. É guardar dados numéricos de forma própria para cálculo.

- O formato é conhecido pelo Python: **shape**
- Os dados ficam organizados para operações rápidas
- Operações podem ser aplicadas ao array inteiro, sem escrever um **for** manual

Com NumPy, a operação deixa de ser item por item no código e passa a agir sobre o bloco numérico inteiro.

⚙ Por que NumPy?

Listas são flexíveis, mas **lentas** quando temos milhares ou milhões de números.

A biblioteca **NumPy** oferece o tipo **array**, especializado em cálculos numéricos.

Vantagens:

- Operações **elemento a elemento** sem **for**
- Muito mais rápido que listas para grandes volumes
- Funções prontas para vetores e matrizes

Como instalar e importar:

```
# No terminal:  
# pip install numpy  
  
import numpy as np
```

Hoje é uma **introdução leve**. Vamos aprofundar em aulas mais à frente.

Criando arrays

Código:

```
import numpy as np

v = np.array([1, 2, 3, 4, 5])
print(v)
print("Tipo:", type(v))
print("Shape:", v.shape)
print("Tamanho:", v.size)
```

Saída:

```
## [1 2 3 4 5]
```

```
## Tipo: <class 'numpy.ndarray'>
```

```
## Shape: (5,)
```

```
## Tamanho: 5
```

Um **array** 1D tem shape `(n,)`. Um **array** 2D tem shape `(linhas, colunas)`.

⚙️ Operações elemento a elemento

A grande diferença em relação a listas:

Código:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)      # soma elemento a elemento
print(a * b)      # produto elemento a elemento
print(a * 2)      # multiplica todos os elementos por 2
print(a.sum(), a.mean(), a.max())
```

Saída:

```
## [11 22 33]
## [10 40 90]
## [2 4 6]

## 6 2.0 3
```

Compare com listas: `[1,2,3] + [10,20,30]` faz **concatenação**, não soma.

 Código junto: lista vs array

Digite e compare:

```
import numpy as np

lista_a = [1, 2, 3]
lista_b = [10, 20, 30]
print("Listas:", lista_a + lista_b)

array_a = np.array([1, 2, 3])
array_b = np.array([10, 20, 30])
print("Arrays:", array_a + array_b)
print("Dobro:", array_a * 2)
```

Depois altere: calcule `array_a * array_b` e explique o resultado.

Matrizes com NumPy

Código:

```
import numpy as np

M = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(M)
print("Shape:", M.shape)
print("Soma total:", M.sum())
print("Soma por linha:", M.sum(axis=1))
print("Soma por coluna:", M.sum(axis=0))
```

Saída:

```
## [[1 2 3]
##  [4 5 6]]
```

```
## Shape: (2, 3)
```

```
## Soma total: 21
```

```
## Soma por linha: [ 6 15]
```

 Lista vs Array

Característica	Lista Python	Array NumPy
Tipos misturados	Sim	Não, mesmo tipo
Operações vetorizadas	Não	Sim
Velocidade	Menor	Maior
Precisa de <code>import</code>	Não	Sim
Boa para...	Coleções gerais	Cálculo numérico

Nesta aula, **listas resolvem tudo o que precisamos**. NumPy é uma porta de entrada para o que vem depois.

PAUSA PRÁTICA: ARRAYS

 Exercícios da seção: arrays

1. Crie dois arrays $a = [1, 2, 3, 4]$ e $b = [10, 20, 30, 40]$. Mostre $a + b$, $a * b$ e $a * 3$.
 2. Crie um array com as notas $[7.5, 8.0, 6.0, 9.0]$. Mostre média, maior e menor valor.
 3. Crie uma matriz NumPy 2 x 3. Mostre seu **shape**, a soma total, a soma por linha e a soma por coluna.
- Se o NumPy não estiver disponível no computador do aluno, ele acompanha visualmente e resolve a mesma lógica com listas.

EXERCÍCIOS GUIADOS

 Guiado 1: Maior e menor

Tarefa: dada uma lista, descobrir o maior e o menor sem usar `max` e `min`.

```
nums = [12, 5, 8, 23, 1, 17, 4]
```

```
maior = nums[0]
menor = nums[0]
for n in nums:
    if n > maior:
        maior = n
    if n < menor:
        menor = n

print("Maior:", maior)
print("Menor:", menor)
```

```
## Maior: 23
```

```
## Menor: 1
```

 Guiado 2: Filtrar pares

Tarefa: a partir de uma lista, criar outra apenas com os números pares.

```
nums = [3, 8, 11, 4, 7, 2, 6, 9]

pares = []
for n in nums:
    if n % 2 == 0:
        pares.append(n)

print("Original:", nums)
print("Pares:", pares)
print("Quantos pares:", len(pares))
```

```
## Original: [3, 8, 11, 4, 7, 2, 6, 9]
```

```
## Pares: [8, 4, 2, 6]
```

```
## Quantos pares: 4
```

 Guiado 3: Soma de duas matrizes

Tarefa: somar duas matrizes 3x3 elemento a elemento.

```
A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
B = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]
```

```
C = []  
for i in range(len(A)):  
    linha = []  
    for j in range(len(A[i])):  
        linha.append(A[i][j] + B[i][j])  
    C.append(linha)  
  
for linha in C:  
    print(linha)
```

```
## [10, 10, 10]  
## [10, 10, 10]  
## [10, 10, 10]
```


LISTA DE EXERCÍCIOS EM SALA

 Exercícios em sala (1/3)

- 1) Crie uma lista com cinco nomes. Imprima o primeiro, o último e o tamanho da lista.
- 2) Dada a lista `nums = [10, 20, 30, 40, 50]`, imprima `nums[1:4]`, `nums[:3]` e `nums[::-1]`.
- 3) Leia 5 números do teclado e armazene em uma lista. Imprima a soma, a média e o maior.
- 4) Dada uma lista, **inverte-a** sem usar `reverse()` (use fatiamento ou um `for` que percorre do fim ao começo).
- 5) Crie a lista dos números pares de 0 a 50 usando `range` e `for` com `if`.

 Exercícios em sala (2/3)

- 6) Dadas duas listas `a = [1, 2, 3]` e `b = [4, 5, 6]`, calcule a **soma elemento a elemento** sem usar NumPy.
- 7) Crie uma tupla `ponto = (3, 7)`. Use desempacotamento para guardar as coordenadas em `x` e `y`. Em seguida, imprima a distância do ponto até a origem (use `(x**2 + y**2) ** 0.5`).
- 8) Dada a lista `notas = [6.0, 4.5, 9.0, 7.5, 3.0, 8.0]`, conte quantos alunos estão **acima da média** da turma.
- 9) Crie uma matriz 3x3 com a tabuada do 7. Mostre na tela em formato de tabela.
- 10) Dada uma matriz 3x3, calcule a soma de cada linha e imprima cada total separadamente.

 Exercícios em sala (3/3)

11) Dada a matriz abaixo, imprima a **diagonal principal**:

$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

Saída esperada: 1 5 9.

12) Dada uma lista de inteiros, crie outra lista contendo somente os elementos **únicos** (sem repetição). Dica: use `if x not in nova:`
`nova.append(x)`.

13) Crie uma lista de tuplas (**nome**, **nota**) para 5 alunos. Use `sorted` para ordenar pela **nota** em ordem decrescente e imprima o ranking.

14) Dadas as listas **produtos** e **precos**, calcule o valor total da compra (soma dos preços) e o produto mais caro.

15) Dada uma matriz qualquer $n \times m$, calcule e imprima a **média de todos os elementos**.

DESAFIOS EXTRAS

Desafios complementares

D1) **Transposição de matriz:** dada uma matriz **M** de **n** linhas e **m** colunas, gere a matriz transposta **MT** de **m** linhas e **n** colunas.

D2) **Produto escalar com NumPy:** repita o exercício do produto escalar usando `numpy.array` e a operação `*` elemento a elemento. Compare com a versão usando `for`.

D3) **Filtragem com tuplas:** dada uma lista de tuplas (`produto, preco`), gere uma nova lista contendo apenas os produtos com preço acima de R\$ 50.

D4) **Mínimo da segunda coluna:** dada uma matriz de números, descubra o menor valor da **segunda coluna**, sem usar `min` direto sobre a coluna inteira (use um laço).

D5) **Histograma simples:** receba uma lista de números entre 0 e 10 e imprima quantas vezes cada valor aparece, ordenando do mais frequente para o menos frequente.

RESUMO

Resumo da Aula

- **Sequências** são coleções ordenadas com indexação a partir de 0
- **Listas** `[ ]` são mutáveis, com muitos métodos (**append**, **insert**, **remove**, **pop**, **sort**)
- **Fatiamento** `[i:j:k]` extrai pedaços de uma sequência
- **Tuplas** `( )` são imutáveis e ótimas para registros fixos
- **Desempacotamento** atribui múltiplas variáveis em uma linha
- Em Python puro, **vetores** são listas 1D e **matrizes** são listas de listas
- **for** aninhado percorre matrizes linha por linha, coluna por coluna
- **NumPy** oferece arrays especializados, com operações elemento a elemento e maior desempenho
- Funções nativas (**len**, **sum**, **min**, **max**, **sorted**, **enumerate**, **zip**) seguem sendo nossas amigas

📖 Referências Bibliográficas

- FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson, 2016.
- MENEZES, N. N. C. **Introdução a Programação com Python**: Algoritmos e Lógica de Programação para Iniciantes. 4. ed. Rio de Janeiro: Novatec, 2024.
- Documentação Python, seção **Data Structures**: docs.python.org/3/tutorial/datastructures.html
- Documentação NumPy, **Quickstart**: numpy.org/doc/stable/user/quickstart.html

Obrigado!

Prof. Dr. José Jairo de Santana e Silva

Universidade Positivo - 2026/1